第三届（2021）集成电路 EDA 设计精英挑战赛

赛题指南

一． **赛题五**：带约束的智能版图拼接

二． **命题企业**：北京华大九天科技股份有限公司

三． 赛题 **Chair：** 林亦波 北京大学计算机系助理教授

四． 赛题背景：

在 IC 的设计的过程中，常常需要遇到如下版图拼接的情形：

通常在 IC 设计的角落位置，已经摆放了其他器件，在剩余的空间区域内，需要摆放一些电路元件，即对于给定数目的多个不同形状和尺寸的芯片版图，根据给定规则，对其进行合理摆放，使模块可以在限定区域摆放成功，并且满足模块布线的要求。

而模拟 IC 模块的设计难以摆脱人工，因有以下挑战：

1. 结构复杂、参数、设计规则以及指标多，搜索空间庞大
2. 器件级的拓扑结构与布线复杂多变
3. 极度依赖人的经验积累、研发周期长

人工版图拼接的工作量负担沉重，因为其结果好坏的指标计算时间很长，设计迭代周期较长，而且手动拼接利用率也难达最优，急需自动生成版图拼接技术。同时问题也可扩展到其他有关 AI Placement 的应用，前景十分广泛。

五． 赛题描述：

1）输入：给定 N 个不同形状和尺寸的多边形(相当于版图边界外形)，多边

形为矩形或边均为正交方向的多边形，给定摆放的多边形限制区域以及模块之间的端口连接关系。（判定多边形在限定区域是否可以布线成功由华大提供接口支持。华大将提供 20000 个样本，每个样本包括所有器件的位置和方向，以及模块的配对端口，以及布线是否成功的相关指标等）

2）求：限定区域内的模块拼接结果

3）拼接规则：

a) 各多边形之间不允许重叠（overlap）

b) 各多边形允许做基本的几何旋转（0°/90°/180°/270°/MX/MY/MXR90/MYR90）

c) 摆放多边形不可以超出限定的区域，否则视为无效多边形

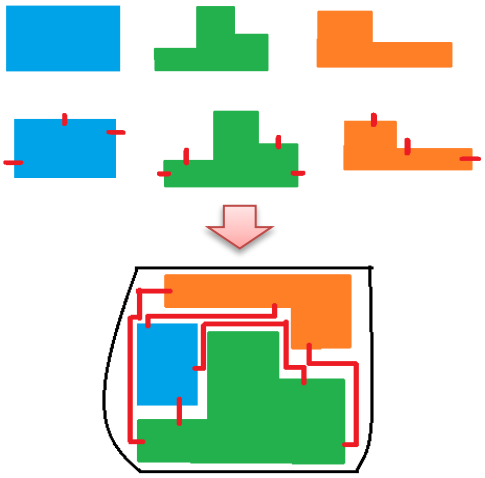
d) 多边形拼接的有效性，需通过布线接口验证是否可以布线成功

4）加分项

同时给出限定区域内的多边形布局和布线结果，布线结果通过华大布线 DRC 规则检查

5）举例：

对以下 3 个多边形，如下的拼接方式（虽然必不可少也会造成一些面积浪费），但是在限定的区域内摆放所有多边形，布线成功，总线长相对较短。



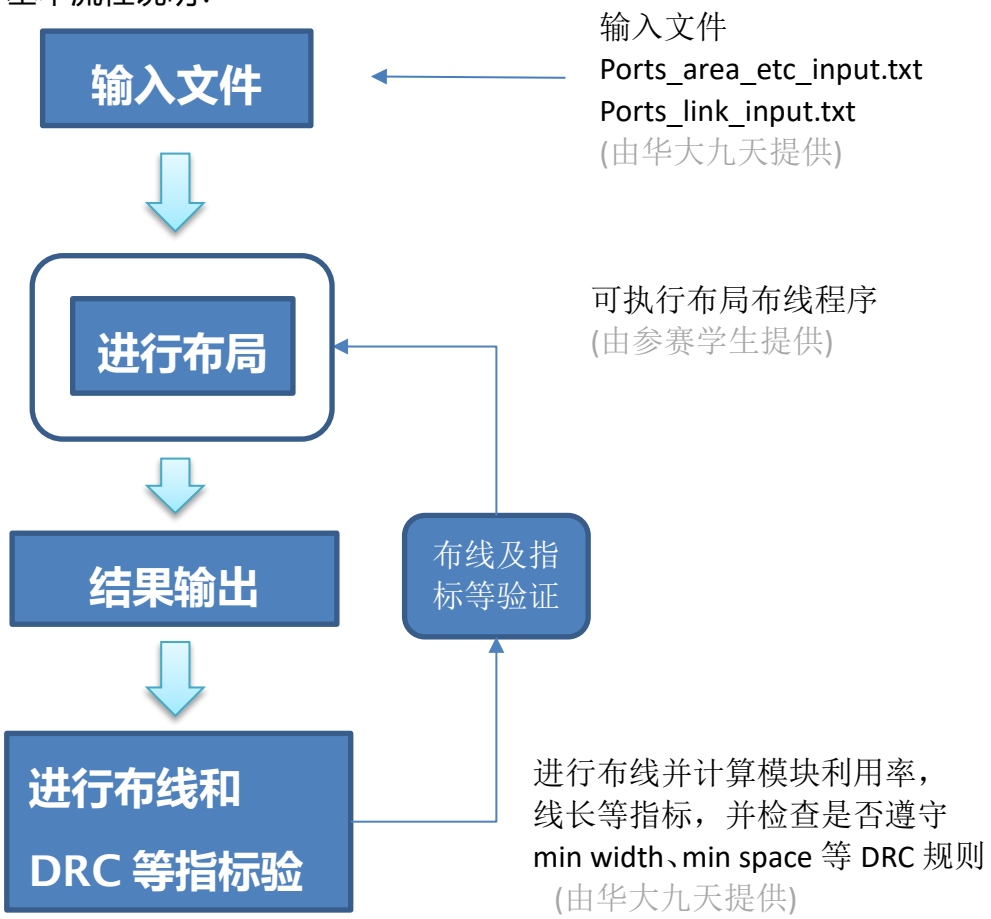
6）布线器布线规则说明：

a) 有连接关系的模块尽量放在一起

b) 端口距离较近的优先连接

c) 其他的布线在剩余空间搜索布线路径，如果布线失败，引擎将受影响的布线拆掉，采取拆线的策略进行重新布线，如果尝试一定次数，如果布线还是不成功，宣告失败，结果以布线成功条数多的结果为准。

7）基本流程说明：



最优布局的算法总体求解流程如上，打分流程将对输出结果进行验证，并输出布局的结果指标，评分按照打分规则进行。

8）格式文件说明：

模块文件格式 Ports_area_etc_input_1.txt：

```
Area: (XXX, XXX) (XXX, XXX) (XXX, XXX) (XXX, XXX)     // 布线区域设置
Rule: SD (5, 5); GATE (4, 4); SD_GATE (0.5); SD_ITO (0.5); GATE_ITO (0.5)
Module：M1                                            //模块 M1
Boundary：(0, 0) (90, 0) (90, 90) (0, 90); GATE       //M1 的边界
Port:  (0, 45) (5, 45) (5, 50) (0, 50); SD            //M1 的第一个端口，颜色 SD
Port:  (45, 0) (45, 5) (50, 5) (50, 0); GATE          //M1 的第二个端口，颜色 GATE
Module：M2
Boundary：(XXX, XXX)  (XXX, XXX)  (XXX, XXX)  (XXX, XXX); ITO
Port:  (XXX, XXX) (XXX, XXX) (XXX, XXX) (XXX, XXX); SD
Port:  (XXX, XXX) (XXX, XXX) (XXX, XXX) (XXX, XXX); GATE
```

连接文件格式 Ports_link_input_1.txt：

```
Link1:
M1    M2    M3
1     1     3    //模块 M1 的第一个端口，与 M2 的第一个端口和 M3 的第三个端口连接在一起
Link2:
M5    M7
2     3          //模块 M5 的第二个端口，与 M7 的第三个端口连接在一起
Link3:
M3    M5
1     2
```

摆放输出模块位置文件格式 result_1.txt：

```
Module：M1                                          //模块 M1
Orient：R0/R90/R180/R270/MX/MY/MXR90/MYR90          //M1 摆放位置的旋转方向
Offset :( XXX, XXX)                                 //M1 摆放的中心点位置
Module：M2
Orient：R0/R90/R180/R270/MX/MY/MXR90/MYR90
Offset :( XXX, XXX)
…
```

9）提交要求：

1. 源工程项目代码
2. 服务器上可编译执行程序prai.exe
3. 一份设计报告，说明算法设计和测试效果。

其中，对可执行程序 prai.exe 的要求是：

a) Linux 服务器环境上可以正确执行

b) 执行示例： prai.exe /xxx/xxx/Ports_area_etc_input.txt /xxx/xxx/Ports_link_input.txt

c) 输出：/xxx/xxx/result.txt

d) 说明：

- 输出文本在用户特定目录下
- 输出文本命名有要求，必须是 result.txt

10）求解思路猜想：

a) 爬山、局部搜索、蚁群算法、遗传算法、模拟退火等

b) 强化学习

c) 数学建模方法

d) 图卷积网络

e) 参考文献，见本文最后章节

六．评分标准

a）布局结果：每个 case 基本分 100 分

（1）全部放入限定区域并且布线成功 70 分

不能全部放入，分数等于 70 分*模块利用率*模块面积利用率

模块利用率 = 成功模块数/总模块数量

模块面积利用率 = 成功模块面积/总模块面积

说明：如果在限定区域放入较多模块，但是通过布线接口检查，发现有些多边形接口连接失败，失败的模块将不计入成功个数

和面积，如果摆放模块存在重叠，重叠模块都将不计入利用率结果。

（2）全部成功放入模块布线的总线长按照排序打分，总线长越短分数越高， 20 分/17 分/15 分/13 分/11 分/9 分/7 分/5 分/3 分/1 分， 其他零分

（3）全部成功放入的模块重心按照排序打分，重心越低分数越高，10 分/9 分/8 分/7 分/6 分/5 分/4 分/3 分/2 分/1 分，其他零分

b）运行时间：20 分

（1）每个 case 5 分钟内出结果，不扣技术分。每超过 1 分钟，扣 4 分，扣分上限 20 分。

（2）10 分钟内不出结果，技术得分判 0

（3） 线程不超过 8 个

c）九天将根据评分规则的不足，逐步改进规则，以利于提升比赛水平

七．参考资料

1) TR Lin，D Penney，M Pedram，L Chen: A Deep Reinforcement Learning Framework for Architectural Exploration: A Routerless NoC Case Study. 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)

2) A Mirhoseini，A Goldie，M Yazgan，JW Jiang，J Dean: A graph placement methodology for fast chip design. 10.1038/s41586-021-03544-w

3) A Goldie，A Mirhoseini: Placement Optimization with Deep Reinforcement Learning. 2020

4) Azalia Mirhoseini, Anna Goldie, Google：Chip Placement with Deep Reinforcement Learning. 22 Apr 2020. https://arxiv.org/abs/2004.10746v1

5) Otten, R.; Ginneken, L. P. P. P.: Floorplan Design Using Simulated Annealing. Prov. Of the ICCAD 1984, 96-98, 1984

6) Ritter, H.; Schulten, K.: Topology Conserving Mappings for Learning Motor Tasks. in Neural Networks for Computing, AIP Conference Proceedings 151, Ed. J.S.Denker, pp.376-380, Snowbird, Utah, 1986

7) Zhang, C.; Mlynski, D. A.: VLSI-placement with a neural network model. Proc. Int. Symp. on Circ. and Syst. , pp. 475-478, 1990.

8) Aykanat, C.; Bultan, T.; Haritaoglu, I.: A fast neural-network algorithm for VLSI cell placement', NeuralNetworks, Vol. 11, No. 9, pp.1671–1684. 1998

9) Hopfield, J.; Tank, D.W.: Neural computation of decisions in optimization problems

10) H .; F u ji y o s h i, K .; K a jit a n i, Y .: VLSI module placement based on rectangle packing by the sequence-pair,'' IEEE Trans. on Computer-Aided Design of Integrated Circuits and Sys-tems,

Vo1.15, No.12, pp.1518-1524 (1996).

11) MCNC Benchmark Netlists for Floorplanning and Placement [Online] https://s2.smu.edu/~manikas/Benchmarks/MCNC_Benchmark_Netlists.html

12) Maegan Tucker, Ellen Novoseller, Preference-Based Learning for Exoskeleton Gait Optimization, 26 Sep 2019, https://arxiv.org/abs/1909.12316

13) https://xueshu.baidu.com/usercenter/paper/show?paperid=1p6n04c01a1h0640nh2606r0rk025842&site=xueshu_se

14) https://zhuanlan.zhihu.com/p/31067515

15) H Kong，T Yan，MDF Wong：Automatic bus planner for dense PCBs. Design Automation Conference, 2009. DAC '09. 46th ACM/IEEE